Live Cell Painting (LCP): full pipeline, from acquisition to Go/No-Go

HCI/HCA Course — hands-on tutorial, from cookiecutter to the phenotypic profiling pipeline

NanoCell Interactions Lab

2026-08-03

Table of contents

This tutorial walks through the Live Cell Painting (LCP) pipeline end to end: creating the project, organizing metadata, running CellProfiler (AssayDev → Analysis), and, from there, running the Python/marimo analysis pipeline up to the Go/No-Go decision. The goal is that you can follow along by copying and pasting the commands, in the order they appear.

Before you begin

- If you don't yet know what Live Cell Painting is and why it exists, start with the Live Cell Painting — History and Overview lesson. This tutorial is the practical part — it assumes the conceptual part has already been covered.
- You need CellProfiler with the RunCellpose plugin installed — see Install CellProfiler & Cellpose.
- You need Pixi and Git installed. We also recommend VS Code — see Install VSCode if you haven't used it yet.
- We recommend opening this tutorial in one browser tab and keeping the terminal open in another window. In the HTML version, every code block has a copy button.

0.1 Pipeline overview

The pipeline has two quite distinct halves, which talk to each other through one folder (`workspace/backend/`):

```
Acquisition (Cytation 5)
    → Metadata (platemap + LoadData)
    → AssayDev (segmentation QC)
    → Analysis (feature extraction)
    → backend/ (CSV / SQLite)
    → NB01 → NB07 (marimo pipeline)
    → Go / No-Go
```

Stage	Tool	What it produces
Acquisition	Cytation 5	raw images per well/site/channel
Metadata	Load Data Generator + Layout Generator	`load_data.csv`, `platemap/`, `barcode_platemap.csv`
AssayDev	CellProfiler (`assaydev.cppipe`)	segmentation tested on 1 site/well, QC montages
Analysis	CellProfiler (`analysis.cppipe`)	features for every cell, exported to `backend/`

Stage	Tool	What it produces
Analysis pipeline	Python + marimo (`hca_pipeline`)	normalized profiles, PCA/UMAP/LDA, quality metrics, Go/No-Go decision

Every stage lives in its own folder under `workspace/`, all organized by the same experiment identifier (`EXPERIMENT_ID`). That's what guarantees you always know where to look for a given file, no matter which stage of the pipeline you're looking at.

0.2 Creating the project with `cookiecutter-lcp`

If you don't have a project yet, install Cookiecutter (once, on your machine):

```
pip install --user pipx
pipx install cookiecutter
cookiecutter --version
```

Generate the project from the lab's template:

```
cookiecutter gh:labnanocell/cookiecutter-lcp
```

Cookiecutter will ask a few fields. The first two are required; the rest are optional and help compose the repository name:

```
author_name (Your Name): Marcelo Bispo
author_email (you@example.com): bispo@unicamp.br
org_name (nanocell-interactions-lab-unicamp):
modality (lcp):
project_tag (nanotoxicology):
cell_models (): Huh7
tissue ():
perturbation_category ():
nanoparticle (): npps
drug ():
genetic ():
```

The repository name (`repo_name`) is assembled automatically from these fields — for example `lcp-nanotoxicology-huh7-npps`. It is not the name of a single experiment: it's the umbrella repository that will accumulate every experiment in your graduate project.

> **Choose `repo_name` carefully**
>
> Renaming the repository later is strongly discouraged (it affects the GitHub link, REDU, and anything you've already shared). Think of it as `<modality>-<focus>-<model>` and check that it describes the whole project, not just the first experiment.

This creates the structure below (with no experiment inside it yet):

```
<repo_name>/
├── images/
├── workspace/
│   ├── metadata/
│   │   └── templates/                # barcode_platemap.csv,
platemap_metadata.txt
│   ├── load_data_csv/
│   ├── pipelines/
│   │   └── templates/                # assaydev.cppipe, analysis.cppipe
│   ├── hca_pipeline/                  # shared Python package (config, io,
normalize, ...)
│   ├── assaydev/
│   ├── segmentation/
│   │   └── cellpose/
│   │       ├── model/                 # already-trained segmentation models
│   │       └── training/
│   ├── analysis/
│   │   └── templates/                 # marimo notebooks 01→07
│   ├── profiles/
│   ├── backend/
│   └── models/
├── workspace_dl/
└── pixi.toml
```

Notice that no folder is per-experiment yet. The `templates/` subfolders are permanent library material — every time you start a new experiment, you'll copy from them, not edit them directly.

`hca_pipeline/` and `segmentation/cellpose/{model,training}/` are not per-experiment either: they exist once and are shared by every experiment in the repository.

Once that's done, initialize Git and create the remote repository:

```
cd <repo_name>
git status
git add .
git commit -m "Initial scaffold via cookiecutter-lcp"

git remote add origin <URL-of-your-repo-on-GitHub>
git branch -M main
git push -u origin main
```

Install the computational environment (Pixi handles every Python dependency — `marimo`, `pandas`, `scikit-learn`, `pycytominer`, `copairs`, etc.):

```
pixi install
pixi run check
```

`pixi run check` runs `scripts/check_environment.py`, which simply tries to import every required package. If it prints `Environment check passed.`, you're good to go.

0.3 Creating a new experiment

Every experiment gets an `EXPERIMENT_ID` in the format:

```
YYYY_MM_DD_CellLine_Treatment_Condition
```

Example: `2025_06_28_Huh7_NPPS_24h`. Set the environment variable and create that experiment's folder structure, copying the contents of each `templates/`:

```
EXP=2025_06_28_Huh7_NPPS_24h

# images and (optional) illumination correction
mkdir -p images/$EXP/images
```

```
mkdir -p images/$EXP/illum

# metadata — copied from templates/, you'll edit it next
mkdir -p workspace/metadata/$EXP/platemap
cp workspace/metadata/templates/* workspace/metadata/$EXP/

# .cppipe pipelines — copied from templates/, you'll adapt them next
mkdir -p workspace/pipelines/$EXP
cp workspace/pipelines/templates/* workspace/pipelines/$EXP/

# segmentation QC and the real segmentation output
mkdir -p workspace/assaydev/$EXP/outlines_qc
mkdir -p workspace/segmentation/cellpose/objects/$EXP

# analysis notebooks — copied from templates/
mkdir -p workspace/analysis/$EXP/analysis
cp workspace/analysis/templates/* workspace/analysis/$EXP/analysis/

# heavy outputs (never pushed to GitHub) and profiles
mkdir -p workspace/backend/$EXP
mkdir -p workspace/profiles/$EXP
mkdir -p workspace/models/$EXP

mkdir -p workspace_dl/$EXP/notebooks
```

Confirm with `tree -L 4` (or `find . -maxdepth 4`) that the structure looks as expected before moving to the next step. Everything else in this tutorial uses this same `EXPERIMENT_ID`.

> ⚠ Avoid renaming once created
>
> Once you've started filling in metadata and running CellProfiler against this `EXPERIMENT_ID`, renaming it later means fixing paths in several places (`.cppipe`, `load_data.csv`, `barcode_platemap.csv`). It's worth double-checking the name before moving on.

0.4 Preparing metadata

This step creates the files that will replace CellProfiler's first four modules (`Images`, `Metadata`, `NamesAndTypes`, `Groups`) with a single `LoadData` module — far more reproducible and much less error-prone.

0.4.1 1. Load Data Generator

The lab uses two programs (shortcuts on the lab computers) designed for Live Cell Painting on the Cytation 5:

- Load Data Generator (LDG) — generates the files with each image's location and minimal metadata;
- Layout Generator — converts the platemap into experimental metadata files.

Running the LDG:

1. Select the folder containing the experiment's images (e.g. `images/$EXP/images/250808_101736_AONPPS_Pro`
   Do not rename folders or files after this step — CellProfiler will lose the reference.
2. The program recognizes channels from the image file names. In Live Cell Painting, the convention is:
   - GFP → AOGFP
   - Propidium Iodide → AOPI
   - DAPI (if used) → Nuclei
3. Click OK. It runs quickly and closes, creating a `load_data_csv/` subfolder inside the image folder.

If you have biological replicates run on different days, repeat this once per day/folder.

The generated `load_data_csv/` folder contains three files:

File	Contents
`load_data.csv`	main reference: channels, wells, sites, plates
`load_data_with_illum.csv`	only used if there is illumination correction (rare on the Cytation 5)
`metadata_representative_cells.csv`	used later to identify representative images

After confirming the three files were generated correctly, move (or rename per replicate,

e.g. `load_data_csv_R1/`) that content into `workspace/load_data_csv/$EXP/` in your project.

0.4.2 2. Platemap and Layout Generator

The Layout Generator needs a platemap: a table where each interior well contains exactly 4 fields, separated by a single space, in this order:

```
CellLine Treatment Concentration Type
```

Valid example: `Huh7 NPPS 1000 trt`.

Field	Description	Example	Note
1. Cell line	cell line name	`Huh7`	always the same format — avoid `HUH7`, `Huh-7`
2. Treatment	compound/nanoparticle	`NPPS`	no spaces (`NonTreated`, not `Non treated`)
3. Concentration	numeric value	`1000`	no units — document units separately
4. Type	well role	`trt`, `negcon`, `poscon`	lowercase, no spaces

The first row and first column of the table stay blank (they represent the plate borders). Wells that were used only to adjust acquisition ("sacrificial wells") must be left blank rather than filled with the 4 fields — otherwise the analysis will try to reference images that don't exist.

> ⚠ The Layout Generator is sensitive to inconsistency
>
> `HUH-7 Polystyrene_0.42um C-1-0.1g/L trt` will not be recognized — the output spreadsheet comes back without your data. Keep the 4-field, single-space pattern rigorous.

Step by step in the Layout Generator:

1. Enter the number of wells on the plate (e.g. `96`).
2. Select the platemap file.
3. Fill in the labels for each unique name found (`Cell_Type`, `Treatment`, `Concentration`, `Control_Type`).

4. Choose your project's `metadata/` folder as the output. The program creates a `platemap/` subfolder with the converted layout (`layout_dayN.csv` and `.txt`).

If the experiment spans several independent days, repeat once per day.

0.4.3 3. `barcode_platemap.csv`

This file is created manually and links each physical plate (imaged on a specific day) to its platemap:

Assay_Plate_Barcode	Plate_Map_Name
`220505_095645_Plate_1`	`layout_day1.txt`
`220614_143057_Plate_1`	`layout_day2.txt`

- `Assay_Plate_Barcode` = the name of the image folder for that replicate.
- `Plate_Map_Name` = the name of the layout file generated in the previous step.

Save it as `workspace/metadata/$EXP/barcode_platemap.csv` (the template already includes an example under `workspace/metadata/templates/` — adapt it instead of starting from scratch).

By the end of this step you should have, inside `workspace/metadata/$EXP/`:

```
workspace/metadata/2025_06_28_Huh7_NPPS_24h/
├── barcode_platemap.csv
└── platemap/
    ├── layout_day1.csv
    └── layout_day1.txt
```

0.5 Loading metadata into CellProfiler

Open CellProfiler in the environment with plugins (Cellpose included) — see Install CellProfiler & Cellpose if you haven't set it up yet.

```
conda activate cp_cellpose
cellprofiler
```

> **Warning**
>
> On Windows, stay on the `C:` drive to open CellProfiler — opening it from another drive (e.g. `D:`) tends to error out.

Copy `assaydev.cppipe` and `analysis.cppipe` from `workspace/pipelines/$EXP/` (you already copied them from `templates/` when you created the experiment) and open `assaydev.cppipe` first:

1. Add the LoadData module (+ → search "LoadData"). CellProfiler will ask whether you want to remove `Images`, `Metadata`, `NamesAndTypes`, and `Groups` — confirm with OK.
2. Under Input data file location, point to the folder containing `load_data.csv` (not the file itself).
3. Select the `load_data.csv` file. Use the View button to confirm channels, wells, sites, and plates were read correctly.
4. Recommended settings:
   - Load images based on this data? → `Yes`
   - Rescale intensities? → `Yes`
   - Base image location → confirm it points to the real image folder (adjust if it's set to `Default Input Folder`)
   - Select metadata tags for grouping → usually `Well`, `Site`, `Plate`
   - Process just a range of rows? → `No` (only use `Yes` to test on a subset)

Test the setup by adding a temporary `IdentifyPrimaryObjects` module, choosing `AOGFP` as the input image, and running Start Test Mode. You should see a table with `FileName_AOGFP`, `FileName_AOPI`, `Metadata_Well`, `Metadata_Site`, and `Metadata_Plate` correctly populated.

0.6 AssayDev — segmentation quality control

AssayDev tunes segmentation parameters on a single site per well, before running the whole experiment. The goal is a segmentation that is *reasonable and consistent* — not perfect. Occasional, isolated errors (e.g. a "giant cell" from a low cytoplasmic signal-to-noise ratio) aren't a big concern and can be filtered out later; systematic errors, which repeat with the same pattern, are what actually deserve attention, because they introduce bias.

> **Keep nuclei/cells that touch the image border**
>
> Excluding border objects tends to make neighboring cells "claim" those nuclei as part of their own cytoplasm, distorting the result. It's safer to keep them and filter afterward with `FilterObjects` if needed.

In `assaydev.cppipe`, the main modules (already configured in the template, but review each one) are:

Module	Function
`FlagImage`	selects 1 site per well (`Site` metadata, min/max range for the desired site)
`RunCellpose`	segments nuclei from `AOGFP`, using a pretrained model → `NucleiCP` object
`FlagImage` (2)	discards images with fewer than 3 nuclei (unrepresentative field)
`Smooth`	smooths the image before cell segmentation
`IdentifySecondaryObjects`	segments the whole cell from `NucleiCP` → `Cells` object
`IdentifyTertiaryObjects`	cytoplasm = `Cells` − `NucleiCP` → `Cytoplasm` object
`RescaleIntensity` + `ImageMath`	equalizes GFP/PI intensities, helps segment nucleoli/vesicles
`IdentifyPrimaryObjects` (nucleoli, vesicles)	nucleoli inside the nucleus; vesicles in the cytoplasm, PI channel
`OverlayOutlines` + `SaveImages`	saves QC images (outlines + montages)

Running it:

1. Clear whatever images were loaded in the pipeline and add this experiment's.
2. Run Update on the `Metadata/LoadData` module and confirm the extraction makes sense.
3. Run Start Test Mode and review each module.
4. Run the full pipeline (`assaydev.cppipe`, still just 1 site/well).
5. Evaluate the output folder — if segmentation is *good enough*, move the QC images to `workspace/assaydev/$EXP/outlines_qc/`.

AssayDev checklist

- ☐ Folders and metadata configured
- ☐ `LoadData` working (tested)
- ☐ `FlagImage` filtering exactly 1 site per well
- ☐ Nucleus/cell/cytoplasm/nucleolus/vesicle segmentation tuned
- ☐ QC via `SaveImages` working (montages + outlines)

0.7 Analysis — feature extraction across the whole plate

With segmentation validated in AssayDev, it's time to run the full analysis. The goal is not a "perfect" pipeline — a pipeline perfected for one specific image is likely overfit and will fail to generalize to others.

1. Open `analysis.cppipe` (the base pipeline, already with `LoadData` configured) in a second CellProfiler window, keeping `assaydev.cppipe` open as a reference.
2. Drag the segmentation modules from `assaydev.cppipe` into `analysis.cppipe`, right after `LoadData`: `RunCellpose`, `Smooth`, `IdentifySecondaryObjects`, `IdentifyTertiaryObjects`, `RescaleIntensity`, `ImageMath`, `EnhanceOrSuppressFeatures`, `MaskImage`, `IdentifyPrimaryObjects`. You don't need the 1-site-per-well `FlagImage` — now you want every image.
3. Confirm `analysis.cppipe`'s `LoadData` points to this experiment's same `load_data.csv`.
4. Make sure measurements (`ExportToSpreadsheet` / `Measure*` modules) are being taken on the raw images (`AOGFP`, `AOPI`), not on intermediate images created only to help segmentation.

Tracing an object's flow

Right-click an analysis module and choose Trace inputs/outputs. It shows where each piece of data comes from and where it's used — useful for confirming nothing got disconnected while copying modules between pipelines.

Configure CellProfiler to save output as a SQLite database (or CSVs, if you prefer) inside `workspace/backend/$EXP/`. Enable, under `File > Preferences`, the "Save pipeline and/or file list in addition to project" option = `Pipeline`, so you don't have to remember to save manually.

Once it finishes running, you should have, in `workspace/backend/$EXP/`, a `.sqlite` database (or a set of `.csv` files) containing, for every segmented cell, hundreds of shape, intensity, and

texture features, organized by three *compartments*:

Object	Description
`Cells`	the whole cell
`Cytoplasm`	the region between nucleus and cell boundary
`Nuclei`	the nucleus

0.8 From CellProfiler to the analysis pipeline

From here on, everything happens in Python — the notebooks read directly from that database/CSV in `workspace/backend/$EXP/`.

Internally, CellProfiler produces one file per compartment (`Cells.csv`, `Cytoplasm.csv`, `Nuclei.csv` — or equivalent tables inside the `.sqlite`). `01_samples_retrieval.py` (the pipeline's first notebook) automatically does what used to be manual:

- prefixing each feature with its compartment of origin (`Cells_AreaShape_Area`, `Nuclei_AreaShape_Area`, ...), so features with the same name coming from different objects don't get confused;
- joining the three compartments into a single per-cell table;
- consolidating every plate/replicate of the experiment into one file: `single_cell_profiles.parquet`.

You don't need to write this code — that's exactly NB01's job. From here, the tutorial moves inside the Pixi environment.

0.9 Pixi environment and marimo notebooks

The analysis notebooks are not Jupyter — they're marimo notebooks: plain reactive `.py` files (`app = marimo.App()`), with no hidden cell state.

Edit a notebook (opens the reactive editor in your browser):

```
pixi run marimo edit workspace/analysis/$EXP/analysis/01_samples_retrieval.py
```

The widgets at the top of each notebook (experiment picker, thresholds, checkboxes) are `mo.ui` elements — changing one automatically re-runs every cell that depends on it.

Run a notebook headlessly (deterministic, every `mo.ui` falls back to its default value — this is

what you'd use for "just run the whole pipeline" or in CI):

```
pixi run python3 workspace/analysis/$EXP/analysis/01_samples_retrieval.py
```

> **A marimo naming detail**
>
> A variable name starting with `_` is private to the cell where it was created. If you edit a notebook and need a variable to be read by a different cell, the name must not start with `_` — otherwise `marimo check --fix` silently drops it from that cell's exports.

0.10 Walking through the pipeline: NB01 → NB07

Notebook	Input	Output	What it does
`01_samples_retrieval.py`	CellProfiler CSV/SQLite	`single_cell_profiles.parquet`	Ingests and consolidates profiles, structural sanity checks (SC-01→SC-05)
`02_aggregate_normalize_feature_select.py`	`single_cell_profiles.parquet`	`well_features_selected.parquet`	Annotates with the platemap, aggregates to well level, normalizes (`mad_robustize`), selects features (SC-06→SC-10)
`03_phenotypic_profiling.py`	`well_features_selected.parquet`	PCA/UMAP/LDA figures and models	PCA, UMAP, LDA with leave-one-plate-out CV, Cohen's d, KMeans, uncorrected-vs-Harmony comparison (SC-11→SC-13)

Notebook	Input	Output	What it does
`04_phenotypic_fingerprints.py`	`per_well_features_selected.parquet`	fingerprints by feature taxonomy	feature taxonomy, per-condition Cohen's d, heatmaps/radar plots — dose axis optional
`05_quality_metrics.py`	`per_well_features_selected.parquet`	quality metrics + decision	Percent Replicating, mAP, batch assessment, dose-response, Go/No-Go decision (SC-18→SC-22)
`06_single_cell_analysis.py`	`single_cell_ready.parquet` (NB02 cache)	SHAP, classification report, figures	single-cell PCA/UMAP, HDBSCAN+KMeans, LightGBM+SHAP, Mahalanobis distance (SC-14→SC-17)
`07_recovery_axis_analysis.py` (optional)	`per_well_features_selected.parquet`	recovery axis	geometric axis between two reference states — only applies to experimental designs with that structure; skips itself automatically if not configured

Run them in order, reviewing the widgets at the top of each before moving to the next:

```
pixi run python3 workspace/analysis/$EXP/analysis/01_samples_retrieval.py
pixi run python3
workspace/analysis/$EXP/analysis/02_aggregate_normalize_featureselect.py
pixi run python3 workspace/analysis/$EXP/analysis/03_phenotypic_profiling.py
pixi run python3 workspace/analysis/$EXP/analysis/04_phenotypic_fingerprints.py
```

```
pixi run python3 workspace/analysis/$EXP/analysis/05_quality_metrics.py
pixi run python3 workspace/analysis/$EXP/analysis/06_single_cell_analysis.py
```

Every notebook imports from one shared Python package, `hca_pipeline` (`workspace/hca_pipeline/`) — it's what knows how to read `experiment_config.json`, apply normalization, compute quality metrics, and so on. You don't need to edit that package to run the pipeline on a new experiment; it's written to be dataset-agnostic.

0.11 Quality metrics and the Go/No-Go decision

NB05 is the pipeline's quality gate. It summarizes three classic image-based profiling metrics:

Metric	Question it answers	Range	Good signal	Reference
PR (Percent Replicating)	What fraction of treatments replicate above null?	0–1	> 0.25	Caicedo et al. 2020
PM (Percent Matching)	What fraction of profiles match the same treatment?	0–1	> 0.50	Caicedo et al. 2020
mAP (Mean Average Precision)	How well can we retrieve replicates from the ranking?	0–1	> 0.50, FDR-significant	Kalinin et al. 2025

And applies the decision criteria:

Check	ID	Critical?	Pass condition
Control validation	SC-19	Yes	Negcon PR ≤ 0.10 and poscon PR > 0.50
PR fraction	PR-25%	Yes	≥ 25% of treatments pass PR per plate

Check	ID	Critical?	Pass condition
Batch effect	SC-20	No	< 30% PR drop cross-plate (only when meaningful)
Dose-response	SC-21	No	monotonic dose-response for multi-dose treatments (skipped if no dose axis)
Phenotype detectability	SC-22	Yes	poscon mAP > 0.5, all treatments show a phenotype

`NB06` reads NB05's Go/No-Go report and warns if the experiment received a No-Go — worth reviewing that warning before interpreting any single-cell result.

> **!** If the result is No-Go
>
> A No-Go doesn't necessarily mean a lost experiment: it usually points at something specific — poorly separated controls, a plate effect dominating the signal, or a concentration/time-point outside the detectable range. Go back to NB05's report before deciding whether the experiment needs repeating or the analysis just needs adjusting.

0.12 Final checklist

- ☐ Project created with `cookiecutter-lcp`, `pixi install` + `pixi run check` passing
- ☐ `EXPERIMENT_ID` defined and folders created from `templates/`
- ☐ Metadata ready: `barcode_platemap.csv` + `platemap/` filled in
- ☐ AssayDev reviewed and QC saved to `workspace/assaydev/$EXP/outlines_qc/`
- ☐ Analysis run, data in `workspace/backend/$EXP/`
- ☐ NB01→NB06 run in order, reviewing each notebook's widgets
- ☐ NB05's Go/No-Go report read and understood

0.13 Where to go from here

- Already have a `per_well_features_selected.parquet` ready (from another pipeline) and want to jump straight to NB03? See the bonus tutorial *Starting the analysis from per_well_features_selected.parquet*.
- Curious about future extensions of the mAP framework (plate-effect diagnostics, batch diagnostics, dose diagnostics) and about SHAP failure modes on imaging data: these are research notes still in development in the lab, outside the scope of this introductory tutorial.
- Core references: Caicedo et al. (2020), *Nature Methods*, doi:10.1038/s41592-020-0851-3; Kalinin et al. (2025), *Nature Communications*, doi:10.1038/s41467-025-60306-2; Cimini et al. (2023), *Nature Protocols*, doi:10.1038/s41596-023-00840-9.